

RUBY

Надёжность · Конкурентность · Runtime · Память

Команда 5

Тимошенко Дарья
Андреев Ярослав
Гиниятуллина Аида



Ярослав



Дарья



Аида

03 презентация

Ruby: надёжность и выполнение программы

- обработка ошибок и проверка данных
- безопасная работа с ресурсами
- конкурентность и потоки
- runtime Ruby, память и GC

ruby

```
1 value = 42
2 puts "Result: #{value}"
```

OUTPUT

Result: 42

Где мы находимся в третьей части

- 01 — базовый синтаксис и коллекции
- 02 — функции, ООП и структура программы
- 03 — надёжность и runtime
- ошибки → ресурсы → потоки → память

ruby

```
1 def positive(value)
2   value >= 0 ? [:ok, value] : [:error, "negative"]
3 end
4
5 puts positive(12).inspect
```

OUTPUT

```
[:ok, 12]
```

Надёжная Ruby-программа

- 1. Ошибки — exception или контролируемый результат
- 2. Проверки — условия и контракты
- 3. Ресурсы — блоки автоматически закрывают файлы
- 4. Конкурентность — Thread и синхронизация
- 5. Runtime — VM, память и GC

ruby

```
1 def divide(a, b)
2   raise ZeroDivisionError if b == 0
3   a / b
4 end
5
6 puts divide(10, 2)
```

OUTPUT

5

Как Ruby представляет ошибки

- Exception — для аварийных ситуаций
- nil — когда значения может не быть
- Result-подход — ошибка как обычное значение
- тип и структура результата делают обработку явной

ruby • блок 1

```
1 def find_item(id)
2   id == 1 ? "Ruby" : nil
3 end
4
5 puts find_item(1)
```

OUTPUT

Ruby

ruby • блок 2

```
1 def parse_number(text)
2   Integer(text)
3 rescue ArgumentError
4   nil
5 end
6
7 puts parse_number("123")
```

OUTPUT

123

ruby • блок 3

```
1 def validate_number(n)
2   n >= 0 ? [:ok, n] : [:error, "negative"]
3 end
4
5 puts validate_number(-5).inspect
```

OUTPUT

[:error, "negative"]

Исключения: `begin` / `rescue` / `ensure`

- `raise` — явно сообщить об ошибке
- `begin` — выполнить потенциально опасный код
- `rescue` — обработать исключение
- `ensure` — выполнить завершающее действие

ruby

```
1 def parse_positive(text)
2   value = Integer(text)
3   raise ArgumentError, "negative" if value < 0
4   value
5 end
6
7 begin
8   puts parse_positive("-7")
9 rescue ArgumentError => e
10  puts e.message
11 ensure
12  puts "finished"
13 end
```

OUTPUT

```
negative
finished
```

Ошибка как значение: Result-подход

- `[:ok, value]` — успешный результат
- `[:error, message]` — контролируемая ошибка
- ошибка обрабатывается без отдельного `rescue`
- подход удобен для ожидаемой валидации

ruby

```
1 def parse_score(text)
2   score = Integer(text, exception: false)
3
4   if score && (0..100).cover?(score)
5     [:ok, score]
6   else
7     [:error, "score out of range"]
8   end
9 end
10
11 result = parse_score("95")
12 puts result.inspect
```

OUTPUT

```
[:ok, 95]
```

Валидация данных через Result

- проверяем формат входной строки
- отдельно проверяем диапазон значения
- ошибка содержит понятную причину
- результат можно передать дальше по цепочке

ruby

```
1 def validate_score(text)
2   score = Integer(text, exception: false)
3
4   return [:error, "not a number"] unless score
5   return [:error, "score > 100"] if score > 100
6   return [:error, "negative score"] if score < 0
7
8   [:ok, score]
9 end
10
11 puts validate_score("85").inspect
12 puts validate_score("120").inspect
```

OUTPUT

```
[:ok, 85]
[:error, "score > 100"]
```

Проверки и контракты

- `raise ArgumentError` — проверка входных данных
- `raise` — проверка состояния программы
- постусловие можно проверить после вычисления
- проверки делают ошибку ближе к месту возникновения

ruby

```
1 def average(values)
2   raise ArgumentError, "empty" if values.empty?
3   raise "negative value" unless values.all? { |v| v >= 0 }
4
5   result = values.sum.to_f / values.length
6   raise "invalid result" if result < 0
7   result
8 end
9
10 puts average([80, 90, 100])
```

OUTPUT

90.0

Безопасная работа с ресурсами

- File.open принимает блок
- после блока файл закрывается автоматически
- ошибка внутри блока не отменяет закрытие
- меньше ручного управления ресурсом

ruby

```
1 File.open("demo.txt", "w") do |file|
2   file.puts "one"
3   file.puts "two"
4   file.puts "three"
5 end
6
7 lines = File.open("demo.txt") do |file|
8   file.each_line.map(&:chomp)
9 end
10
11 puts lines.inspect
12 File.delete("demo.txt")
```

OUTPUT

```
["one", "two", "three"]
```

Конкурентность и параллелизм

- Thread позволяет выполнять задачи независимо
- конкурентность — задачи продвигаются независимо
- параллелизм зависит от runtime и доступных ресурсов
- для общего состояния нужна синхронизация

ruby

```
1 threads = 2.times.map do |i|
2   Thread.new do
3     sleep 0.1
4     "task #{i + 1} done"
5   end
6 end
7
8 puts "tasks started"
9 puts threads.map(&:value).sort
```

OUTPUT

```
tasks started
task 1 done
task 2 done
```

Future-подобное ожидание результата

- в Ruby нет встроенного Future
- Thread может представлять вычисление, которое завершится позже
- value ждёт завершения и возвращает результат
- это простой способ показать идею асинхронного результата

ruby

```
1  def slow_square(x)
2    Thread.new do
3      sleep 0.2
4      x * x
5    end
6  end
7
8  future = slow_square(5)
9  puts "started"
10 puts future.value
```

OUTPUT

```
started
25
```

Сборка нескольких конкурентных задач

- каждая задача возвращает свой Thread
- value собирает готовый результат
- несколько задач можно запустить до ожидания
- результаты объединяются в обычную коллекцию

ruby

```
1  def load_name(id)
2    Thread.new { "student-#{id}" }
3  end
4
5  def load_score(id)
6    Thread.new { 70 + id }
7  end
8
9  name = load_name(3)
10 score = load_score(3)
11
12 puts "#{name.value}: #{score.value}"
13
14 threads = [1, 2, 3].map { |id| load_score(id) }
15 puts threads.map(&:value).inspect
```

OUTPUT

```
student-3: 73
[71, 72, 73]
```

Гонки данных и Mutex

- общий изменяемый var создаёт риск гонки
- `counter += 1` состоит из нескольких шагов
- Mutex делает критическую секцию взаимно исключающей
- ещё лучше — уменьшать shared mutable state

ruby

```
1 counter = 0
2 mutex = Mutex.new
3
4 threads = 10.times.map do
5   Thread.new do
6     100.times do
7       mutex.synchronize { counter += 1 }
8     end
9   end
10 end
11
12 threads.each(&:join)
13 puts counter
```

OUTPUT

1000

Ruby runtime: что происходит под капотом

- .rb → парсер/VM → выполнение
- MRI использует YARV bytecode
- RubyVM::InstructionSequence позволяет увидеть инструкцию
- детали зависят от реализации Ruby

ruby

```
1 code = "x = 1 + 2"
2 iseq = RubyVM::InstructionSequence.compile(code)
3
4 puts iseq.class
5 puts iseq.disasm.lines.first(5)
```

OUTPUT

```
RubyVM::InstructionSequence
далее — инструкции YARV
```

Память: объекты и ссылки

- переменная хранит ссылку на объект
- две переменные могут ссылаться на один объект
- изменение объекта видно через обе ссылки
- это важно при работе с mutable state

ruby

```
1 a = +"Ruby"  
2 b = a  
3  
4 b << " memory"  
5  
6 puts a  
7 puts b
```

OUTPUT

```
Ruby memory  
Ruby memory
```

Garbage Collector и управление памятью

- Ruby автоматически управляет памятью
- GC находит объекты, которые больше не нужны
- GC.start можно запросить вручную
- точный момент сборки контролирует runtime

ruby

```
1  objects = 100_000.times.map { Object.new }
2  puts "objects created"
3
4  objects = nil
5  GC.start
6
7  puts "garbage collection requested"
```

OUTPUT

```
objects created
garbage collection requested
```

Эффективный стиль Ruby

- не удерживать ненужные объекты
- использовать Enumerable и блоки
- lazy-цепочки не создают промежуточные коллекции
- меньше shared mutable state —
проще конкурентный код

ruby

```
1 result =  
2   (1..1_000_000)  
3     .lazy  
4     .select { |n| n.even? }  
5     .map { |n| n * n }  
6     .first(3)  
7  
8   puts result.inspect
```

OUTPUT

```
[4, 16, 36]
```

Итоговый пример: безопасный конвейер

- парсинг входных строк → Result
- проверка данных → raise
- вычисление среднего → postcondition
- два отчёта → конкурентные задачи
- один пример объединяет основные идеи

ruby

```
1 def parse_line(line)
2   name, score_text = line.split(",", 2)
3   return [:error, "invalid line"] unless name && score_text
4
5   score = Integer(score_text, exception: false)
6   return [:error, "invalid score"] unless score && (0..100).cover?(score)
7
8   [:ok, { name: name, score: score }]
9 end
10
11 input = ["Ann,90", "Bob,75", "Cara,82"]
12 parsed = input.map { |line| parse_line(line) }
13
14 raise "input error" if parsed.any? { |r| r[0] == :error }
15
16 students = parsed.map { |r| r[1] }
17 raise "empty data" if students.empty?
18
19 average = students.sum { |s| s[:score] }.to_f / students.length
20 raise "invalid average" unless average >= 0
21
22 reports = [
23   Thread.new { "average=#{average}" },
24   Thread.new { "passed=#{students.count { |s| s[:score] >= 60 }}" }
25 ]
26
27 puts reports.map(&:value).inspect
```

OUTPUT

```
["average=82.33333333333333", "passed=3"]
```